

Design Note — LLD Practice Platform

Full reasoning, alternatives considered, and rejected options for every decision below live in `DECISIONS.md`. This note pulls that reasoning into the shape the assignment brief asks for.

MVP

A learner picks one of 5 seeded LLD problems (Parking Lot, Elevator, Vending Machine, Movie Ticket Booking, Rate Limiter), each with real requirements text, a scope boundary, and a rubric weight map reflecting what actually matters for that problem (e.g. Vending Machine weights class-responsibilities and abstraction higher, since its central challenge is state handling). The learner works in a textarea pre-filled with a six-section Markdown template (Requirements & Assumptions, Classes & Responsibilities, Relationships & Interactions, Key Behaviour & Rules, Edge Cases & Testability, Trade-offs & Extensibility, plus an optional mermaid diagram section). On submit, two evaluators run in parallel behind the same contract: a real rule-based evaluator scoring 3 of 8 rubric dimensions deterministically, and an AI evaluator (OpenAI `gpt-5-nano`) scoring all 8 with evidence citations, against a strict JSON schema. The result screen polls until both reach a terminal state and shows per-criterion feedback with quoted evidence, top priorities, and a clearly separate “automated checks” panel for the rule-based results. History shows per-dimension score deltas against the learner’s previous attempt on the same problem, and flags dimensions that stay weak across different problems.

User flow

`Choose problem → work in the template → submit → poll for feedback → review → try again` — the exact loop the brief names, mapped one-to-one onto the four screens (problem list, problem detail/editor, result, history).

Important classes and interfaces

The domain is plain TypeScript classes with private fields and explicit `interface` ports — real classes with behavior, not data bags:

- **Attempt** (aggregate root) — owns the practice lifecycle (`Draft → Submitted`), holds at most one `Submission`, enforces the three-case idempotency rule on `submit()`.
- **Submission** (immutable value object) — the frozen design document at submit time: raw content, format, parsed `DesignDocument`, content hash.

- **Evaluation** (entity, its own lifecycle: Pending → Running → Completed | Failed) — one row per evaluator per attempt, tagged with which evaluator produced it and which rubric version it was scored against.
- **Feedback** (immutable value object held by Evaluation) — summary, per-criterion scores/evidence/concern/suggestion, domain-computed topPriorities and aggregateScore.
- **DesignDocument** — the format-neutral parsed shape (addressed sections + code blocks + optional diagram source) that everything downstream operates on; nothing past the adapter boundary knows what Markdown is.
- **Rubric** — owns the 8 dimensions, their named anchors, per-problem weights, and aggregate(), including the guard that returns no score at all when not every dimension was scored (so a partial rule-based evaluation never shows a misleading total).
- **Four ports:** FormatAdapter (Markdown adapter today), Evaluator (start(...)/report-back via EvaluationRecorder.record(...), implemented by AiEvaluator, RuleBasedEvaluator, and a FakeEvaluator used only in tests), Repository (four focused interfaces: Learner/Problem/Attempt/Evaluation, backed by hand-written SQLite mappers, no ORM), LlmClient (the OpenAI adapter).
- **EvaluationOrchestrator** (application service) — the only place that fans out to N evaluators, owns the timeout timer per evaluation, and is the single inbound path (record()) through which any evaluator's outcome re-enters the domain.

Evaluation approach

Every rubric dimension has 5 named, described anchors (0 Missing → 4 Exemplary) rather than a raw 0–10 score — a model given a definition is measurably more consistent than one given a vibe, and this is the single biggest lever on consistency available without a fine-tuned model. The AI evaluator returns per-criterion judgements only; the domain computes the aggregate and the top 2–3 priorities by lowest weighted contribution — the model never invents a total, inverting the assignment's named anti-pattern of "an LLM prompt that simply asks for a 100-point score." Every evidence citation is a structured quote-plus-location, and the domain deterministically verifies each quote actually appears in the submitted document before trusting it; a fabricated quote gets dropped and that criterion's confidence is down-ranked, rather than failing the whole evaluation. The rule-based evaluator scores the 3 dimensions that are genuinely checkable without judgement (requirement-coverage length, edge-case count, explanation-marker presence) and explicitly abstains on the other 5 — the abstention is deliberate, not a placeholder: it demonstrates, by running code rather than by assertion, which parts of evaluation are actually deterministic.

Key trade-offs

- **Markdown over code or diagram-only submission** — one encoding carries prose, fenced code, and mermaid diagrams; headings give stable addresses for citations. Rejected freeform text (nothing

deterministic to check, no anchors for evidence) and rendered-diagram upload (UI-heavy, and the assignment penalizes UI-heavy/logic-light submissions).

- **A real second evaluator, not a fallback branch** — resilience comes from the evaluator model itself (two evaluators always run), not from a special-cased recovery path. If the AI evaluation fails, the learner still has the rule-based result, because it was already running, not because a fallback fired.
- **In-process background execution, no message broker** — satisfies “don’t block the submission request” without a distributed-systems build. The honest cost (a crashed process strands `Running` evaluations) is paid explicitly with a startup sweep to `Failed("interrupted")`, in a few lines, rather than hidden.
- **No ORM** — `Submission` and `Feedback` are immutable value objects; an ORM entity is the opposite of that. Hand-written mappers keep the domain persistence-ignorant, which is a point being made deliberately, not a corner being cut.

Two Change Tests

Test A — today text, later a class diagram. Register one new `FormatAdapter` implementation that parses diagram content into the same `DesignDocument` shape (same addressed-section keys, same diagram slot). **Domain model changes: zero.** Nothing past the adapter boundary — the rubric, the evaluators, the citation verification — knows or cares what format produced the `DesignDocument` it’s given.

Test B — today one evaluator, later a rule-based (or human) one. It’s not a future feature — it’s already running in production. `RuleBasedEvaluator` and `AiEvaluator` both implement the same `Evaluator` port and are registered together (`[AiEvaluator, RuleBasedEvaluator]`) as the permanent N=2 in `server.ts`. Adding either one required registering a class against an existing contract and nothing else — the practice flow, the orchestrator, and the API were never touched. A human-review adapter would slot in identically: `start()` creates a review task in an inbox, `record()` is called hours later when a person submits it.

The HLD line

The first component to extract, if this needed to scale beyond a single process, is the **evaluation worker** — it is the only part of the system with a different resource profile (LLM latency, per-call token cost) and a different failure mode (timeouts, malformed structured output, provider rate limits) than the rest of the monolith. Every other component (API routes, repositories, the orchestrator’s bookkeeping) is fast, synchronous, and local; the evaluation worker is the one piece that is slow, external, and unreliable, which is exactly the profile that justifies giving it its own deployment unit and its own queue once the current in-process/pollled-status approach stops being enough.

Limitations, stated deliberately

- **No authentication.** Identity is a declared handle (typed once, persisted client-side) — not a secure session. History only needs a stable learner identity, not a verified one, and the assignment doesn't evaluate authentication anywhere. The seam for a real identity provider sits entirely at the edge; no domain code assumes otherwise.
- **In-process background execution, not a message broker.** Evaluators run as fire-and-forget async work inside the same process as the API, with a startup sweep to recover evaluations stranded mid-run by a crash. This is an explicit, cheap trade-off for a 2-day prototype — a real production system with meaningful evaluation volume would want the evaluation worker (see HLD line above) as its own process with a real queue in front of it.
- **The rule-based evaluator scores only 3 of 8 rubric dimensions** and abstains on the rest, by design — it is not a lesser AI evaluator, it is a deliberately-scoped deterministic one. An evaluation where not every dimension was scored exposes no aggregate total at all, so a partial score is never presented as if it were complete.
- **The History screen's per-dimension delta only compares an attempt to the learner's immediately preceding attempt on the *same* problem**, not a smoothed trend across many attempts — a deliberate scope decision given a 2-day prototype rather than an oversight.
- **No AI-usage cost or rate-limit dashboard.** A provider outage, quota exhaustion, or rate limit will surface as a `Failed` evaluation with a stored, displayed reason (never silently swallowed) — the rule-based evaluator's parallel result is still available in that case — but there is no operator-facing monitoring beyond that.